

Database — how models STORE data, how they CONNECT, how a row is BORN

THE canonical answer to: “kaunsa model data kaise save karta hai, backend me kaise store hota hai, sab kaise jude hain, aur flow kya hai.” Quick ER sketch: [PROJECT KNOWLEDGE MAP §6](#). Per-model one-liners: each app's [GUIDE](#). Yeh file = poora picture + flow diagrams.

0) Pehle base — model = table, instance = row

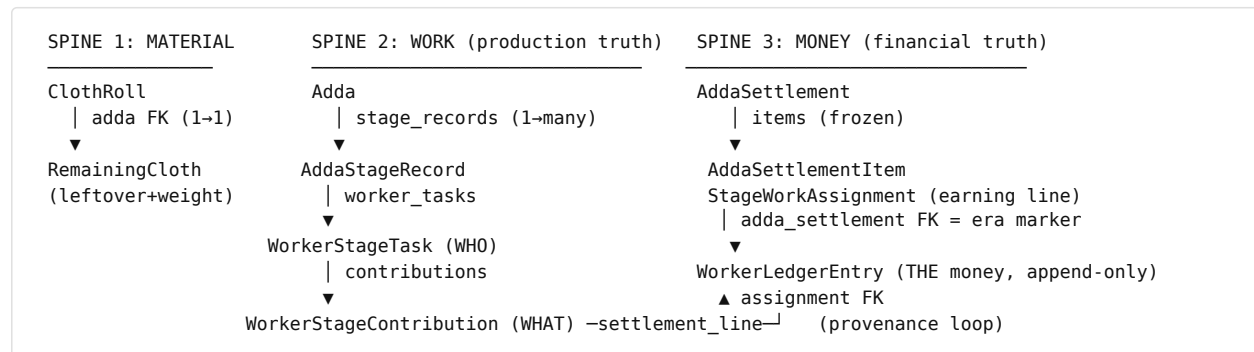
Django me `class Foo(models.Model)` = PostgreSQL ki ek TABLE. Us class ka har object = ek ROW. Har field = ek COLUMN.

```
class WorkerAdvance(models.Model):    → TABLE expense_workeradvance
    amount = DecimalField(...)        → COLUMN amount NUMERIC(12,2)
    worker = ForeignKey(User, ...)    → COLUMN worker_id INTEGER → FK
```

Table naam default = `<app>_<modelname>` lowercase. FK column = `<field>_id` (Postgres sirf ID store karta hai, poori row nahi — JOIN se milti hai). Money HAMESHA `DecimalField` (Float kabhi nahi — paisa exact). `created_at/updated_at` har table me `TimeStampedModel` (core, abstract) se aate hain.

`.save()` → Django `INSERT` (nayi row) ya `UPDATE` (purani) SQL chalata hai. Is project me yeh VIEWS nahi karte — sirf SERVICES, `@transaction.atomic` ke andar (rule 4). Truth/money tables me to har `.save()` single-writer service se hi (rule 5, CI gates).

1) System ke 3 SPINE



Material = raw_materials app · work = production app · money = expense app. Tracking app har spine ke events ka append-only timeline rakhta hai (kisi ko import nahi karta — primitive).

2) Har important model — kya store + example ROW

PRODUCTION TRUTH (PRODUCTION APP)

Model → table	Key columns	Example row
Adda → <code>production_adda</code>	<code>code</code> unique editable=False (auto), <code>product_id</code> , <code>current_stage_id</code> , <code>status</code> , <code>started_at</code> (auto), <code>completed_at</code>	3-PATTI-001, 3 Patti, stage=Cutting, in_progress
AddaStageRecord → <code>production_addastagerecord</code>	<code>adda_id</code> , <code>workflow_stage_id</code> , <code>started_at</code> , <code>completed_at</code> , frozen <code>processing_cost</code> (honest=NULL)	3-PATTI-001, Cutting, processing_cost=NULL until priced
WorkerStageTask (WST) → <code>production_workerstagetask</code>	<code>stage_record_id</code> , <code>worker_id</code> , <code>status</code> , <code>*_at</code> stamps, <code>verified_by</code>	Cutting#9, utest, status=completed
WorkerStageContribution (WSC) → <code>production_workerstagecontribution</code>	<code>task_id</code> , <code>color_id</code> , <code>size_id</code> , <code>reported_quantity</code> (IMMUTABLE), <code>verified_quantity</code> (mgmt fix), frozen <code>expected_rate</code> / <code>expected_earning</code> , <code>settlement_line_id</code> , <code>bundle_item_id</code>	task=..., Red, Size-1, reported=60, verified=NULL, rate=3.00, earning=180.00

MATERIAL (RAW_MATERIALS APP)

Model → table	Key columns	Example
ClothRoll → <code>raw_materials_clothroll</code>	<code>roll_id</code> (CR-seq global), type/color/location FKs, <code>purchased_date</code> , <code>cost_per_kg</code> (PURCHASE fact, nullable), <code>adda_id</code> (1→1), <code>status</code>	CR-000142, Cotton/Red, ₹200/kg, adda=3- PATTI-001, used
RemainingClothOfClothRoll → <code>production_remainingclothofclothroll</code>	<code>roll_id</code> , <code>source_adda_id</code> , <code>remaining_weight_kg</code> , <code>is_consumed</code> , <code>consumed_in_adda_id</code>	leftover 3.10kg of CR- 000142

FINANCIAL TRUTH (EXPENSE APP)

Model → table	Key columns	Example
WorkerLedgerEntry → expense_workerledgerentry	worker_id, entry_type (credit/debit), category (stage_earning/advance_recovery/settlement_payment/reversal...), amount (CHECK >0), entry_date, FK assignment / advance / settlement, reverses_id (self-FK unique)	utest, credit, stage_earning, 180.00, assignment=SWA#42
StageWorkAssignment (SWA) → expense_stageworkassignment	stage_record_id, worker_id, color/size, allocated_quantity, frozen earning_rate_snapshot / earning_amount_snapshot, adda_settlement_id (NULL=era-A / set=era-B), voided_at	Cutting#9, utest, 60, rate=3, amt=180, ADST-0003
AddaSettlement → expense_addasettlement	reference (ADST-XXXX unique), adda_id, status, frozen totals, supersedes_id, settled_at/by	ADST-0003, 3- PATTI-001, final- ized, expected_total=225
AddaSettlementItem → expense_addasettlementitem	FROZEN per-worker: expected_earning, advance_outstanding_before, advance_recovered, final_payable	ADST-0003, utest, expected=225, recovered=0, payable=225
PayrollSettlement / Item → expense_payrollsettlement*	cash payment (SETL-XXXX) + recovery lines (XOR-parented, reversed_at)	SETL-0012, utest, amount_paid=225
WorkerAdvance → expense_workeradvance	worker_id, amount, advance_date, entered_by	loan ₹500 to utest

HISTORY (TRACKING APP — APPEND-ONLY)

AddaHistory/ClothRollHistory/ProductHistory → tracking_*history: change_type, actor, metadata (JSONB),
timestamp. BarcodeBatch → (adda,color,size) + start_seq / end_seq = piece identity ranges.

3) FK rules — kaun kisko delete se bachata hai

on_delete = "parent delete hua to child ka kya":

- **PROTECT** (money+history default): parent delete = ERROR. Ledger ke peeche ka worker/Adda/SWA kabhi gayab nahi — woh ANCHOR hai.
- **CASCADE**: child bina parent bekaar (LayeringRollEntry apne SR ke saath marta).
- **SET_NULL**: optional reference; parent ja sakta, child NULL ho jaata.

Rule: paisa/history? → **PROTECT**.

4) Ek ROW kaise BANTI hai (backend write-flow = “kaise save hota hai”)

```

STEP 1 – worker phone se report
WorkerReportView (assignment-gate)
  ↳ worker_task_service.report_contributions(task, lines)  [@transaction.atomic]
    ↳ WSC.objects.create(...) → INSERT production_workerstagecontribution
      (reported_quantity saved – IMMUTABLE)

STEP 2 – "Submit & Complete"
  ↳ worker_task_service.complete_worker_task(task)
    ↳ task.status=COMPLETED; save() → UPDATE WST
    ↳ har WSC pe expected_rate/earning FREEZE → UPDATE (visibility only)

STEP 3 – owner Adda settle (paisa YAHAN banta hai)
AddaSettlementDetailView(action=finalize)
  ↳ adda_settlement_service.finalize_adda_settlement(...)  [@transaction.atomic]
    advisory 5374 → ADST → stage records → profiles → advances (LOCK ORDER)
    per WSC line:
      ↳ SWA.objects.create(...) → INSERT expense_stageworkassignment
        (amount frozen; adda_settlement set → era-B)
      ↳ WSC.settlement_line = SWA; save() → UPDATE (provenance link)
      ↳ ledger_service.log_credit(assignment=SWA)
        → INSERT expense_workerledgerentry (credit, stage_earning)
    per worker: AddaSettlementItem.objects.create() → INSERT (FROZEN)
    AddaSettlement.status=finalized; save() → UPDATE
    tracking.log_adda(SETTLEMENT_FINALIZED) → INSERT addahistory

STEP 4 – cash day (alag event)
  ↳ settlement_service → ledger_service.log_debit(settlement_payment)
    → INSERT expense_workerledgerentry (debit)

```

Balance kabhi STORE nahi hota — HAR BAAR live SQL `SUM(credit) - SUM(debit)`. Isi liye ledger append-only: galti = ulti REVERSAL row, balance khud sahi.

5) System design + architecture se connection

- **Service layer** (ADR-0001): har INSERT/UPDATE service se → saving predictable, atomic, guarded. Diagram: [04 SERVICE LAYER.md](#).
- **Single-writer** (ADR-0002): har truth-table ka EK writer → CI gates (4/4b/4c) row-integrity guarantee.
- **Production ≠ Financial truth** (ADR-0005): WSC me paisa save nahi hota (expected_* = dikhāva); asli paisa ledger me, settlement par. Do spine isliye.
- **Constraints = DB ki aakhri guard**: `amount>0`, `finalized=settled_at`, `recovered≤outstanding`, partial-unique active task — app bug ho tab bhi Postgres galat row store nahi karega. ([01](#), [03](#))
- Race-safe saving: [03 TRANSACTIONS AND LOCKS.md](#).
- Asli SQL me yeh sab: [09 SQL BEGINNER TO ADVANCED.md](#).

6) Cheap JOIN-walks

`select_related` (FK joins) + `prefetch_related` (reverse sets) read layer (payroll_service); indexes on hot pairs (worker+status, adda+settled_at, task). Perf-baseline tests query-count pin karte hain — N+1 = failing test.

7) Deeper

Field-by-field: app GUIDE + khud `config/<app>/models.py` (har model pe architecture docstring + constraint comments). Domain breakdown: `SYSTEM_DESIGN.md §5`. Core ER: `GLOSSARY.md`.